Probeklausur: Echtzeitsysteme (EZS)

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Allgemeine Hinweise

- **Prüfung:** Semesterabschlussprüfung
- **Bearbeitungszeit:** 90 Minuten
- **Hilfsmittel:** Keine (Closed Book)
- **Maximale Punktzahl:** 100 Punkte

Aufgabe 1: Scheduling-Theorie & RMS (25 Punkte)

Ein neues Steuergerät soll drei periodische Tasks im präemptiven Multitasking verarbeiten. Für die Tasks wurden folgende Parameter (Worst-Case Execution Time C_i, Periode T_i) ermittelt. Es gelten implizite Deadlines ($D_i = T_i$).

- τ_1: $C_1 = 2\,\text{ms}$, $T_1 = 8\,\text{ms}$
- τ_2: $C_2 = 3\,\text{ms}$, $T_2 = 12\,\text{ms}$
- τ_3: $C_3 = 4\,\text{ms}$, $T_3 = 24\,\text{ms}$

1.1 Berechnen Sie die Gesamt-CPU-Auslastung U des Systems. Zeigen Sie Ihren Rechenweg. *(5 Punkte)*

1.2 Vergeben Sie die Prioritäten der Tasks nach den Regeln des *Rate Monotonic Scheduling (RMS)*. Begründen Sie Ihre Wahl in einem Satz. *(5 Punkte)*

1.3 Prüfen Sie mithilfe der Liu-und-Layland-Schranke, ob dieses Task-Set garantiert schedulbar ist. Die Schranke für $n = 3$ lautet $\approx 78{,}0\,\%$. *(5 Punkte)*

1.4 Nennen Sie zwei der strikten mathematischen Annahmen, die Liu und Layland 1973 für diesen Beweis trafen, welche in der Realität jedoch fast nie zu $100\,\%$ erfüllt sind. *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2: Synchronisation & Priority Inversion (20 Punkte)

2.1 Erklären Sie den fundamentalen konzeptionellen Unterschied zwischen einem CMSIS-Mutex und einem binären Semaphor in FreeRTOS. Gehen Sie dabei auf die Begriffe *Ownership* und *Verwendung in Interrupts (ISR)* ein. *(10 Punkte)*

2.2 Beschreiben Sie das Phänomen der *Priority Inversion* (bekannt vom Mars Pathfinder). Welche drei Task-Prioritäten sind daran beteiligt und wie löst das *Priority Inheritance Protocol (PIP)* dieses Problem? *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 3: Multicore & Architektur (20 Punkte)

Sie entwerfen ein System auf dem STM32H755 (Cortex-M7 und Cortex-M4).

3.1 Erklären Sie den Unterschied zwischen SMP (*Symmetric Multiprocessing*) und AMP (*Asymmetric Multiprocessing*). Warum wird für harte Echtzeitsysteme nahezu ausschließlich AMP verwendet? *(10 Punkte)*

3.2 Was ist ein *Cache Coherency Failure*? Beschreiben Sie, wie dieser Fehler auftritt, wenn der M7-Kern über Write-Back-Caches in den geteilten SRAM4 schreibt und der M4-Kern versucht, diese Daten zu lesen. Wie löst man dieses Problem hardwareseitig? *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 4: Safety & Watchdogs (15 Punkte)

4.1 Der STM32 besitzt einen IWDG (*Independent*) und einen WWDG (*Window*) Watchdog. Warum reicht der IWDG nicht aus, um Code vor fehlerhaft übersprungenen Validierungs-Routinen (z. B. durch Bit-Flips) zu schützen? Wie löst das Fenster-Konzept des WWDG dieses Problem? *(8 Punkte)*

4.2 Was ist der EWI (*Early Wakeup Interrupt*)? Nennen Sie eine Aktion, die im EWI-Callback zwingend durchgeführt werden sollte, und eine, die strengstens verboten ist. *(7 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 5: Code-Review (20 Punkte)

Der folgende C-Code soll einen zeitkritischen Hardware-Interrupt auf dem STM32 in FreeRTOS verarbeiten. Das System stürzt jedoch regelmäßig ab oder friert ein.

Finden und benennen Sie **vier** fatale Architektur- oder Programmierfehler in diesem Code-Snippet und erklären Sie kurz, warum diese in einem RTOS tödlich sind. *(Hinweis: Im Snippet sind fünf Fehler angelegt; für volle Punktzahl reichen vier korrekt benannte und begründete Fehler.)*

```
uint8_t data_ready = 0;
osMutexId_t dataMutex;

/* Hardware Interrupt Service Routine (Top-Half) */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    osMutexAcquire(dataMutex, osWaitForever);
    data_ready = 1;
    osMutexRelease(dataMutex);
}

/* RTOS Task (Bottom-Half) */
void ProcessTask(void *argument) {
    for (;;) {
        while (data_ready == 0) {
            /* Warten auf neue Daten */
        }

        data_ready = 0;
        Filter_Complex_Data(); /* dauert ca. 10 ms */

        HAL_Delay(50);
    }
}
```

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)